WESTERN SYDNEY
UNIVERSITY

Reference Variables & Function Overloading

ELEC1006: ENGINEERING COMPUTING

Reference Variable

WESTERN SYDNEY UNIVERSITY

- A mechanism that allows a function to work with the original argument from the function call, not a copy of the argument.
- Allows the function to modify values stored in the calling environment.
- **Provides a way for the function to 'return' more than one value.**

Passing by Reference

WESTERN SYDNEY UNIVERSITY

- A reference variable is an alias for another variable.
- Defined with an ampersand (**&**)

```
void getDimensions(int&, int&);
```

- Changes to a reference variable are made to the variable it refers to.
- Use reference variables to implement passing parameters by *reference.*

The & here in the prototype indicates that the parameter is a reference variable.

WESTERN SYDNEY UNIVERSITY

Example 1

```
1  // This program uses a reference variable as a function
2  // parameter.
3  #include <iostream>
4  using namespace std;
5
6  // Function prototype. The parameter is a reference variable.
7  void doubleNum(int &);
8
9  int main()
10 {
11    int value = 4;
12
13    cout << "In main, value is " << value << endl;
14    cout << "Now calling doubleNum..." << endl;
15    doubleNum(value);
16    cout << "Now back in main. value is " << value << endl;
17    return 0;
18 }
19
```

Here we are passing value by reference.

```
20 //*********************************************************
21 // Definition of doubleNum.                               *
22 // The parameter refVar is a reference variable. The value *
23 // in refVar is doubled.                                  *
24 //*********************************************************
25
26 void doubleNum (int &refVar)
27 {
28    refVar *= 2;
29 }
```

The & also appears here in the function header.

Program Output

```
In main, value is 4
Now calling doubleNum...
Now back in main. value is 8
```

Reference Variable Notes

WESTERN SYDNEY UNIVERSITY

- Each reference parameter must contain **&**.
- Space between type and & is unimportant.
- Must use & in both prototype and header.
- Argument passed to reference parameter must be a variable – cannot be an expression or constant.
- Use when appropriate – don't use when an argument should not be changed by a function, or if a function needs to return only 1 value.

Overloading Functions

WESTERN SYDNEY UNIVERSITY

- Overloaded functions have the same name but different parameter lists.
- Can be used to create functions that perform the same task but take different parameter types or different number of parameters.
- Compiler will determine which version of function to call by argument and parameter lists to pass to the function.

Function Overloading Examples

WESTERN SYDNEY UNIVERSITY

Using these overloaded functions,

```
void getDimensions(int);              // 1
void getDimensions(int, int);         // 2
void getDimensions(int, double);      // 3
void getDimensions(double, double);// 4
```

the compiler will use them as follows:

```
int length, width;
double base, height;
getDimensions(length);              // 1
getDimensions(length, width);       // 2
getDimensions(length, height);      // 3
getDimensions(height, base);        // 4
```

More info

WESTERN SYDNEY UNIVERSITY

[1] cplusplus.com: Variables and types. https://www.cplusplus.com/doc/tutorial/variables/

[2] cplusplus.com: Overloads and templates https://www.cplusplus.com/doc/tutorial/functions2/

[3] learncpp.com: 6.11 – Reference variables https://www.learncpp.com/cpp-tutorial/611-references/comment-page-1/

[4] learncpp.com: 7.6 – Function overloading https://www.learncpp.com/cpp-tutorial/76-function-overloading/